MSR Insight Backend Interview Prep

This document is written for a backend-focused interview. It is based on the actual repository, not generic interview material.

Use the tone in the answers as your speaking style: confident, practical, and honest about tradeoffs.

1. Backend Project Analysis

One-Line Backend Summary

MSR Insight is an academic reporting and proctoring platform where an Express.js gateway manages auth, sessions, database access, scraping, reports, and parent communication, while a FastAPI intelligence service handles AI remarks and RAG-based proctor chat over student academic data.

Actual Backend Architecture

The repo has two backend services:

- `backend/express`: primary **API gateway** and business backend.
- `backend/fastapi`: AI and **RAG** service.

The frontend and Chrome extension consume these services, but your backend story should center on Express, FastAPI, Redis, PostgreSQL, Prisma, Puppeteer, and external AI/reporting integrations.

Runtime Components

Component	Role
Express app	Main **API gateway**, route orchestration, **auth**, **sessions**, scraping, reports
FastAPI app	AI remark generation and **RAG** chatbot
PostgreSQL	Main data store, including **JSONB** student academic data
Prisma	**ORM** used by **Express** for core relational operations
Redis	**Session** store using opaque **session** IDs
Puppeteer + **Cheerio**	Login to college parent portal, scrape academic pages, normalize data
PGVector	**Vector store** for **RAG** chunks inside **Postgres**
Groq	**LLM** provider for AI remarks
Gemini	**Embeddings** and **RAG** chat model in **FastAPI**
Resend	Parent email delivery
Cloudinary	PDF report archival
Twilio WhatsApp	Optional **WhatsApp** dispatch
Chrome extension	Batch scrape trigger for proctor **sessions**

Express Request Lifecycle

1. Request enters `backend/express/server.js`.
2. `server.js` imports `src/app.js` and starts listening on `process.env.PORT || 5000`.
3. `app.js` applies:

- `cors()`
- `express.json()`
- `/api/health`
- route mounts for **auth**, report, proctor, notifications, students, admin
- global error middleware

1. Protected routes call `requireSession`.
2. `requireSession` reads `x-session-id`, checks **Redis**, attaches `req.user`, refreshes the **session TTL**, and continues.
3. Controllers validate basic inputs, call services/repositories, and return JSON.

Express Route Map

Base route	File	Main purpose	Auth status
`/api/auth`	`auth.routes.js`	student/proctor login, register, logout, profile	login/register public; profile/logout check header inside controller
`/api/report`	`report.routes.js`	dashboard data, AI report, manual scrape update, email/**WhatsApp** report	`requireSession` on main routes
`/api/proctor`	`proctor.routes.js`	proctor dashboard, proctee detail, scrape list, notifications, chat	router-level **session** check
`/api/notifications`	`notification.routes.js`	attendance alerts	router-level **session** check
`/api/students`	`students.js`	sync normalized student data	public
`/api/admin`	`admin.routes.js`	proctor **CRUD**, assignments, parent data, stats	public in backend

Important interview detail: the project does not use JWT. It uses Redis-backed opaque session IDs passed in the custom `x-session-id` header.

Authentication And Session Flow

Student login:

1. Frontend sends USN and DOB to `POST /api/auth/login`.
2. `auth.controller.login` validates required fields.
3. `auth.service.login` uses `userRepository.findByCredentials`.
4. DOB is normalized through `formatDOB` inside `userRepository.findByCredentials`.
5. If the student is missing, the service attempts a **Puppeteer** scrape and sync.
6. A **UUID session** ID is generated.
7. **Redis** stores:

- `session:<uuid>` -> `student:<USN>`
- `usn:<USN>` -> `<sessionId>`

1. **Session TTL** is 30 days.

Proctor login:

1. Frontend sends proctor ID and password to `POST /api/auth/proctor-login`.
2. Proctor ID is uppercased.
3. **Prisma** fetches the proctor record.
4. `bcrypt.compare` validates password.
5. Existing **Redis session** is reused if present.

6. Otherwise a new **UUID session** is created.
7. Login triggers `notifyRagSync()` to ask **FastAPI** to sync vector data.

Session middleware:

- Reads `x-session-id`.
- Looks up `session:<id>` in **Redis**.
- Splits identity into role and ID.
- Attaches `req.user`, `req.userId`, and `req.userRole`.
- Refreshes the **session TTL**.

Strong answer to remember:

"I used Redis-backed opaque sessions instead of JWT here. The upside is server-side invalidation and simple logout. The tradeoff is that Redis becomes part of the auth path, so production needs Redis HA, better session key lifecycle, and role/ownership checks."

Authorization Reality

This is a major interview point.

Current backend only verifies that a session exists for many protected routes. It does not consistently verify:

- whether the **session** role is student or proctor,
- whether `req.user.id` matches `:proctorId`,
- whether a student can access only their own `:usn`,
- whether admin routes are backend-protected.

Examples:

- `/api/proctor/:proctorId/dashboard` accepts any valid **session**, but should require a proctor **session** matching `proctorId`.
- `/api/report/:usn` accepts any valid **session**, but should restrict student access to their own USN and proctor access to assigned students.
- `/api/admin/*` is public on the backend; the frontend uses a client-side `admin123` password, which is not production-grade security.
- **FastAPI RAG** endpoints are directly callable from the frontend and trust `proctor_id` from the request body.

Say this honestly in interviews:

"The authentication mechanism is implemented, but authorization is the area I would harden first before production. I would add role-based middleware and ownership checks around proctor, student, report, admin, and FastAPI RAG endpoints."

Database Design

Prisma schema:

- `Student`

- **primary key**: `usn`
- columns: name, dob, phone, email, current_year, details **JSONB**
- relations: parents, proctor maps

- `Proctor`
- **primary key**: `proctor_id`
- password hash
- relations: student maps

- `Parent`
- composite key: `(usn, relation)`
- belongs to student

- `ProctorStudentMap`
- maps proctor to student for an academic year
- unique key: `(student_id, academic_year)`
- enforces one proctor per student per academic year

Why JSONB:

The scraper data is semi-structured and can change when the college portal changes. JSONB gives flexibility for subjects, attendance, CIE marks, and exam history without constantly changing relational tables.

Tradeoff:

JSONB is easy to store and return to the frontend, but harder to query, index, validate, and aggregate. For production analytics, frequently queried fields like attendance percentage, subject code, SGPA, and last sync time should eventually move into relational tables or materialized summaries.

DB Query And Indexing Notes

Good current choices:

- **Primary key** on `students.usn`.
- **Primary key** on `proctors.proctor_id`.
- Composite key on parent `(usn, relation)`.
- Unique constraint on `proctor_student_map(student_id, academic_year)`.

Likely improvement:

- Add composite **index** on `proctor_student_map(proctor_id, academic_year)` because proctor dashboard and notifications query by proctor and academic year.
- Consider **GIN index** or extracted columns for **JSONB** if backend starts filtering by attendance/marks inside `details`.
- Use **transactions** for bulk assignment and multi-step deletes.

Actual issue to revise:

`removeStudent` in `admin.controller.js` calculates `normalizedProctorId` but deletes by `(student_id, academic_year)` only. That can remove a student's assignment even if the URL proctor ID is not the assigned proctor.

Scraping And Data Sync Flow

The scraper is in `puppeteerScraper.service.js`.

Flow:

1. Launch **Puppeteer**.
2. Login to `https://parents.msrit.edu/newparents/`.
3. Collect authenticated cookies.
4. Close the browser.
5. Use **Axios** and **Cheerio** for lighter HTML fetch/parse.
6. Fetch unique attendance, CIE, and exam-history URLs.
7. Normalize:

- subject code/name
- attendance present/absent/remaining/percentage
- T1/T2/AQ marks
- total marks
- current year from semester
- exam history and CGPA

1. Upsert the student into **Postgres** through `studentService.syncStudents`.

Good engineering point:

"The scraper does not keep a full browser open for every page. It uses Puppeteer only for authenticated login, then switches to HTTP requests with cookies, which is lighter and more scalable."

Production concern:

Puppeteer is CPU/memory heavy. At high traffic, scraping should move to a background queue with concurrency limits, retries, timeouts, and persistent job status.

Report Flow

Student dashboard report:

1. `GET /api/report/student/:usn`
2. Check **Postgres JSONB** data first.
3. If data exists with subjects, return DB data.
4. If missing, trigger scraper.
5. Fetch fresh data and return it.

AI remark:

1. `GET /api/report/:usn`

2. Load student dashboard data.
3. Extract `details.subjects`, handling nested compatibility cases.
4. Send payload to **FastAPI** `/generate-remark`.
5. Return AI output and metadata.

Manual update:

1. `POST /api/report/update`
2. Enforces 5-minute cooldown based on `details.last_updated`.
3. Triggers scraper if allowed.
4. Returns updated student data.

Email And WhatsApp Flow

Email:

1. Frontend sends `usn` and `htmlContent` to `/api/report/send-email`.
2. Backend fetches student and parents from **Postgres**.
3. **Puppeteer** renders client-provided HTML into PDF.
4. PDF is uploaded to **Cloudinary**.
5. **Resend** sends the PDF to each parent.

WhatsApp:

1. Frontend sends `usn` and `htmlContent` to `/api/report/send-whatsapp`.
2. Backend generates PDF.
3. Uploads to **Cloudinary**.
4. If **Twilio** is configured, sends **WhatsApp** messages.
5. If not configured, prepares the message payloads.

Production concern:

Rendering arbitrary HTML from the client in Puppeteer should be sanitized or generated server-side from trusted templates. Otherwise it is risky in a production backend.

FastAPI AI Remark Flow

Files:

- `main.py`
- `routers/report_router.py`
- `services/ai_service.py`
- `services/prompt_builder.py`
- `services/llm_provider.py`

Flow:

1. **Express** calls **FastAPI** `/generate-remark`.

2. `report_router.py` accepts the student data as a dict.
3. `AIService` validates `subjects`.
4. `PromptBuilder` builds a strict two-line academic remark **prompt**.
5. `GroqLLMProvider` calls **Groq** using configured model.
6. Response includes student details, AI remark, model, token usage, and generation time.

Interview-safe answer:

"I kept AI remark generation in FastAPI because the Python ecosystem is stronger for AI tooling. Express owns product workflows, while FastAPI owns model calls and RAG logic. That split keeps Node focused on API/business logic and Python focused on LLM orchestration."

FastAPI RAG Flow

Files:

- `routers/rag_router.py`
- `services/rag_service.py`

Sync flow:

1. **Express** proctor login calls `notifyRagSync`.
2. **FastAPI** `/api/rag/sync` starts a background task.
3. `RAGService.sync_data` fetches mapped students from **Postgres** using raw **SQL**.
4. Each student becomes chunks:

- identity
- academics
- attendance
- history
- conduct
- misc

1. **Gemini embeddings** create vectors.
2. Documents are stored in **PGVector** collection `student_data_v2`.

Chat flow:

1. Frontend calls **FastAPI** `/api/rag/chat` directly.
2. Request includes `question` and `proctor_id`.
3. Query is rewritten by **Gemini**.
4. Intent detection selects chunk types.
5. Retrieval combines:

- **BM25** from current proctor documents
- **PGVector semantic retrieval**

1. Retrieved chunks are formatted into context.

2. **Gemini** generates a grounded answer.

Production concerns:

- **FastAPI RAG** endpoint trusts `proctor_id` from the request body.
- No **auth/session** validation on **FastAPI** routes.
- Sync drops **PGVector** tables before adding new documents.
- `is_syncing` is in-memory, so multiple **FastAPI** workers would not share the lock.
- Direct raw **SQL** should be wrapped with tighter connection management and monitoring.

Deployment And Configuration

Current local startup:

- `start-all.bat` and `run.bat` start **FastAPI**, **Express**, and frontend.
- **FastAPI** runs with `python main.py`.
- **Express** runs with `node server.js`.
- Frontend runs with `npm run dev`.

Important gap:

No Dockerfile or docker-compose file exists in the repository.

Say this in interviews:

"Right now it is locally runnable through batch scripts and environment variables. For production, I would containerize Express and FastAPI separately, run migrations in CI/CD, use managed Postgres/Redis, and put both services behind a reverse proxy or API gateway with proper health checks."

Config Shape

Express env keys seen:

- `PORT`
- `FASTAPI_URL`
- `REDIS_URL`
- `DATABASE_URL`
- `RESEND_API_KEY`
- `RESEND_FROM_EMAIL`
- `CLOUDINARY_CLOUD_NAME`
- `CLOUDINARY_API_KEY`
- `CLOUDINARY_API_SECRET`
- `CDP_PORT`
- `CDP_HOST`
- `PIGGYBACK_WHITELIST`

FastAPI env keys seen:

- `GROQ_API_KEY`

- `GROQ_MODEL`
- `GEMINI_API_KEY`
- `DATABASE_URL`
- `PORT`
- `REDIS_HOST`
- `REDIS_PORT`
- `EXPRESS_API_URL`
- `OLLAMA_API_URL`
- `OLLAMA_MODEL`

Frontend env keys:

- `NEXT_PUBLIC_API_URL`
- `NEXT_PUBLIC_FASTAPI_URL`

Do not expose secret values in interviews.

Strong Backend Areas

- **Express** route/controller/service/repository layering.
- **Prisma** schema with relational mapping for students, proctors, parents, assignments.
- **Redis session** management with **sliding TTL**.
- **Puppeteer**-to-**Axios** scraping optimization.
- **JSONB** strategy for changing academic data.
- **FastAPI** service split for AI workloads.
- **RAG** chunking, **metadata filtering**, **BM25** plus vector retrieval.
- PDF generation and parent communication flow.

Weak Backend Areas To Own Honestly

- Admin routes are not backend-protected.
- **FastAPI RAG** routes are unauthenticated.
- No role/ownership **authorization** checks.
- **CORS** is wide open.
- No **API rate limiter** except scraper cooldown.
- No automated test suite.
- No **Docker** production setup.
- Logging is mostly `console.log` / `print`.
- Some legacy files are inconsistent with current **Prisma** schema.
- Bulk operations lack **transactions**.
- Scraping is synchronous and heavy.

2. Complete Interview Conversations

Every follow-up question below includes a follow-up answer. Practice these out loud. The goal is not to memorize every word, but to develop a confident backend story around the actual repository.

Conversation 1: Project Overview

QUESTION:

> **Explain MSR Insight from a backend point of view.**

ANSWER:

MSR Insight is an academic reporting and proctoring system. The backend is split into two services. **Express** is the main business **API**: it handles **authentication**, **Redis sessions**, **Prisma**/**Postgres** access, student and proctor APIs, scraping, report generation, email, and **WhatsApp** workflows. **FastAPI** is the AI service: it generates academic remarks through **Groq** and powers a **RAG** chatbot using **Gemini**, **LangChain**, and **PGVector**. The main backend idea is that **Express** owns the product workflows, while **FastAPI** owns the AI-heavy workflows.

FOLLOW-UP QUESTION 1:

> **What part of the backend did you mainly build or own?**

FOLLOW-UP ANSWER 1:

My main backend contribution was the **API** and data flow. I worked around **auth**, **session** handling, student/proctor routes, **Prisma**/**Postgres** models, scraping and normalization, and the integration with **FastAPI** for AI remarks and **RAG**. I can explain the request lifecycle end to end, from login to dashboard data to report generation.

FOLLOW-UP QUESTION 2:

> **Why is this project backend-heavy?**

FOLLOW-UP ANSWER 2:

Because most of the complexity is behind the UI. The backend has to authenticate users, manage **sessions**, scrape an external portal, normalize messy academic data, store it safely, generate reports, communicate with parents, and call AI services. The frontend displays the result, but the backend coordinates the real workflows.

FOLLOW-UP QUESTION 3:

> **What is the most impressive backend feature?**

FOLLOW-UP ANSWER 3:

The strongest feature is probably the data pipeline: using **Puppeteer** for authenticated portal login, switching to **Axios** and **Cheerio** for lighter page fetching, normalizing that academic data, storing it in **Postgres JSONB**, and then using it for dashboards, reports, alerts, and AI. It is not just **CRUD**; it is an actual backend workflow.

Conversation 2: Backend Architecture

QUESTION:

> **Describe the backend architecture.**

ANSWER:

The architecture is a two-service backend. **Express** is the gateway and product backend. It exposes routes for **auth**, reports, proctors, notifications, student sync, and admin workflows. **FastAPI** is a separate intelligence service for AI remarks and **RAG** chat. **Express** uses **Prisma** to talk to **Postgres**, **Redis** for **sessions**, and **Axios** to call **FastAPI**. **FastAPI** also reads from **Postgres** for **RAG** sync and stores vectors in **PGVector**.

FOLLOW-UP QUESTION 1:

> **Is this a microservice architecture?**

FOLLOW-UP ANSWER 1:

I would not oversell it as pure microservices. It is closer to a distributed monolith or a two-service architecture. **Express** and **FastAPI** are separate runtimes, but they still share the database and are tightly coupled around student data. That is fine for this project stage, but for true microservices I would separate ownership and communication contracts more strictly.

FOLLOW-UP QUESTION 2:

> **Why not keep everything in Express?**

FOLLOW-UP ANSWER 2:

I could keep everything in **Express**, but Python has much stronger AI and **LangChain** tooling. **FastAPI** made it easier to build the **RAG** pipeline, use **embeddings**, and integrate **LLM** providers cleanly. **Express** remains better suited for the main product **API** and Node-based workflows.

FOLLOW-UP QUESTION 3:

> **What is the downside of two services?**

FOLLOW-UP ANSWER 3:

The downside is operational complexity. Now I have two servers, two sets of environment variables, **service-to-service** failures, and deployment coordination. That is why production needs **health checks**, timeouts, private networking, and ideally service **authentication** between **Express** and **FastAPI**.

Conversation 3: Express App Lifecycle

QUESTION:

> **What happens when the Express backend starts?**

ANSWER:

`server.js` imports the **Express** app from `src/app.js` and starts listening on the configured port. In `app.js`, the backend applies **CORS**, JSON parsing, a health endpoint, mounts **auth**, report, proctor, notification, student sync, and admin routes, then registers the global error handler at the end.

FOLLOW-UP QUESTION 1:

> **Why is the error handler at the end?**

FOLLOW-UP ANSWER 1:

In **Express**, error middleware should come after route registration so that `next(error)` from controllers can reach it. If I register it before routes, it will not catch errors from those later route handlers.

FOLLOW-UP QUESTION 2:

> **What middleware runs before your routes?**

FOLLOW-UP ANSWER 2:

The global middleware is `cors()` and `express.json()`. **CORS** allows browser requests, and `express.json()` parses JSON request bodies so controllers can read `req.body`. Route-specific **session** middleware is applied later on protected route groups or individual routes.

FOLLOW-UP QUESTION 3:

> **What would you improve in app startup?**

FOLLOW-UP ANSWER 3:

I would validate required environment variables at startup and fail fast if database, **Redis**, or **FastAPI** config is missing. I would also add a production health endpoint that checks **Postgres** and **Redis**, not just whether **Express** is running.

Conversation 4: Route Organization

QUESTION:

> **Explain your route organization.**

ANSWER:

Routes are grouped by domain. `auth.routes.js` handles login, register, logout, and profile. `report.routes.js` handles dashboard data, AI remarks, update, email, and **WhatsApp**. `proctor.routes.js` handles proctor dashboard, proctee details, scrape list, notifications, and chat. `admin.routes.js` handles proctor management, student assignment, parents, and stats. That separation makes the backend easier to reason about.

FOLLOW-UP QUESTION 1:

> **Which routes are protected?**

FOLLOW-UP ANSWER 1:

Report routes are protected with `requireSession`, and proctor plus notification routes use router-level **session** verification. **Auth** login/register routes are public. Admin routes are currently public on the backend, which is a clear production security gap.

FOLLOW-UP QUESTION 2:

> **Why is route-level organization useful?**

FOLLOW-UP ANSWER 2:

It keeps each **API** surface focused. When I debug report generation, I know to look at report routes, controller, and service. When I debug proctor dashboard issues, I look at proctor routes and repository queries. It avoids one giant controller file.

FOLLOW-UP QUESTION 3:

> **What route cleanup is needed?**

FOLLOW-UP ANSWER 3:

There are two student sync route files, but only `students.js` is mounted. I would remove or merge the duplicate. I would also protect admin routes and make route naming more consistent before production.

Conversation 5: Student Login Flow

QUESTION:

> **Walk me through student login.**

ANSWER:

The student sends USN and DOB to `POST /api/auth/login`. The controller checks required fields. The service normalizes the DOB, looks up the student through **Prisma**, and if the student is missing it tries to scrape and sync the record. Once validated, it creates a random **UUID session**, stores it in **Redis**, and returns the **session** ID and USN to the frontend.

FOLLOW-UP QUESTION 1:

> **Why normalize DOB?**

FOLLOW-UP ANSWER 1:

Because the frontend, database, and portal can represent dates differently. The backend standardizes DOB into `DD-MM-YYYY`, so credential checks do not fail just because one input came as `YYYY-MM-DD`.

FOLLOW-UP QUESTION 2:

> **What happens if the student does not exist?**

FOLLOW-UP ANSWER 2:

The backend tries to scrape the student from the college portal using the provided USN and DOB. If scraping succeeds, it syncs the student into **Postgres** and retries the lookup. If scraping fails, login returns invalid credentials.

FOLLOW-UP QUESTION 3:

> **Is using USN and DOB strong authentication?**

FOLLOW-UP ANSWER 3:

It is acceptable for a college portal-style student lookup, but it is not strong **authentication** for a sensitive production system. I would add stronger **auth** options, rate-limited login, and maybe OTP or institutional SSO if this were going live.

Conversation 6: Proctor Login Flow

QUESTION:

> **Walk me through proctor login.**

ANSWER:

The proctor sends proctor ID and password to `POST /api/auth/proctor-login`. The backend uppercases the ID, fetches the proctor from **Postgres**, compares the password using **bcrypt**, and creates or reuses a **Redis session**. After successful login, it notifies **FastAPI** to start **RAG** sync so the chatbot has fresh student data.

FOLLOW-UP QUESTION 1:

> **Why use bcrypt?**

FOLLOW-UP ANSWER 1:

Passwords should never be stored in plaintext. **Bcrypt** is intentionally slow and salted, so even if password hashes are leaked, brute forcing is harder. It is a standard practical choice for password hashing in a Node backend.

FOLLOW-UP QUESTION 2:

> **Why reuse an existing proctor session?**

FOLLOW-UP ANSWER 2:

It avoids creating unlimited active **sessions** for the same proctor. In this project, one active **session** per proctor is simpler to reason about. The tradeoff is that if we want multi-device **session** management later, we need a more explicit **session** table or per-device **sessions**.

FOLLOW-UP QUESTION 3:

> **Why trigger RAG sync on login?**

FOLLOW-UP ANSWER 3:

The idea is freshness. When a proctor logs in, the backend nudges **FastAPI** to rebuild vectors from the latest student records. For production, I would not tie it only to login; I would sync after data updates or through scheduled/background jobs.

Conversation 7: Session Management

QUESTION:

> **Do you use JWT or sessions?**

ANSWER:

This project uses **Redis**-backed opaque **sessions**, not **JWT**. The backend generates a **UUID session** ID and stores the identity in **Redis** as `session:<id>`. The frontend sends the **session** ID in the `x-session-id` header on protected requests. The middleware validates it and refreshes the **TTL**.

FOLLOW-UP QUESTION 1:

> **Why not JWT?**

FOLLOW-UP ANSWER 1:

JWTs are good for stateless verification, but invalidation is harder. With **Redis sessions**, logout and forced invalidation are straightforward because the server controls the **session** store. The tradeoff is **Redis** becomes part of the **auth** path.

FOLLOW-UP QUESTION 2:

> **How does logout work?**

FOLLOW-UP ANSWER 2:

Logout reads the **session** ID, looks up the identity in **Redis**, deletes the reverse mapping like `usn:<USN>` or `proctor:<ID>`, and deletes `session:<id>`. After that, the same **session** ID will fail middleware validation.

FOLLOW-UP QUESTION 3:

> **What happens if Redis is down?**

FOLLOW-UP ANSWER 3:

Protected routes cannot validate **sessions**, so they should fail closed. In production I would monitor **Redis**, use a managed highly available **Redis** setup, and expose **health checks** so traffic is not routed to an unhealthy instance.

Conversation 8: Session Middleware

QUESTION:

> **What does your session middleware do?**

ANSWER:

`requireSession` reads the `x-session-id` header, checks **Redis** for `session:<id>`, rejects missing or expired **sessions**, splits the identity into role and ID, attaches that to `req.user`, and refreshes the **Redis TTL** to implement a sliding **session** window.

FOLLOW-UP QUESTION 1:

> **Does it check roles?**

FOLLOW-UP ANSWER 1:

Currently it only validates that the **session** exists and attaches the role. It does not enforce route-level roles by itself. That is a gap I would fix by adding middleware like `requireRole('proctor')` and **ownership checks**.

FOLLOW-UP QUESTION 2:

> **What is a sliding TTL?**

FOLLOW-UP ANSWER 2:

It means the **session** expiry is extended whenever the user makes an authenticated request. So active users stay logged in, while inactive **sessions** eventually expire.

FOLLOW-UP QUESTION 3:

> **What is the security risk here?**

FOLLOW-UP ANSWER 3:

A valid **session** is not enough. The backend also needs to verify that the user is allowed to access the route target. For example, a proctor **session** should only access that proctor's students, and a student should only access their own report.

Conversation 9: Authorization Gaps

QUESTION:

> **What is the biggest security weakness in your backend?**

ANSWER:

The biggest weakness is **authorization**. The project has **session** validation, but several routes do not fully enforce role and ownership. Admin routes are public on the backend, **FastAPI RAG** routes are directly callable, and some routes trust URL params or request body values too much.

FOLLOW-UP QUESTION 1:

> **How would you fix proctor authorization?**

FOLLOW-UP ANSWER 1:

I would add middleware that requires a proctor role and checks that `req.user.id` matches `req.params.proctorId`. For student data under a proctor, I would also verify the student is assigned to that proctor for the requested academic year.

FOLLOW-UP QUESTION 2:

> **How would you secure admin routes?**

FOLLOW-UP ANSWER 2:

I would remove the client-side `admin123` flow and implement real backend admin **authentication**. Admins should be stored in the database or handled through SSO, and admin APIs should require an admin **session**, audit logging, and stricter validation.

FOLLOW-UP QUESTION 3:

> **How would you secure FastAPI?**

FOLLOW-UP ANSWER 3:

I would stop exposing **FastAPI** directly to the browser. **Express** should validate the user **session** and then call **FastAPI** internally with a trusted proctor ID. At minimum, **FastAPI** should require a service token or signed internal request.

Conversation 10: Database Schema

QUESTION:

> **Explain your database schema.**

ANSWER:

The core schema has `Student`, `Proctor`, `Parent`, and `ProctorStudentMap`. Students are keyed by USN and store academic data in a **JSONB** `details` field. Proctors have a hashed password. Parents belong to students using a composite key of USN and relation. `ProctorStudentMap` connects students to proctors for a specific academic year.

FOLLOW-UP QUESTION 1:

> **Why use a map table?**

FOLLOW-UP ANSWER 1:

Because proctor assignment is a relationship, not a fixed field on the student. The map table lets us include academic year and change assignments over time without rewriting the student record itself.

FOLLOW-UP QUESTION 2:

> **Why is there a unique constraint on student and academic year?**

FOLLOW-UP ANSWER 2:

It enforces one assigned proctor per student for a given academic year. That matches the business rule and prevents duplicate or conflicting proctor assignments.

FOLLOW-UP QUESTION 3:

> **What schema issue would you improve?**

FOLLOW-UP ANSWER 3:

I would add indexes for the actual dashboard access pattern, especially `(proctor_id, academic_year)` on the mapping table. I would also extract frequently queried metrics from **JSONB** into summary columns or tables.

Conversation 11: JSONB Design

QUESTION:

> **Why store academic details as JSONB?**

ANSWER:

The academic data comes from scraping, so the structure can vary. Subjects, attendance details, assessments, and exam history are semi-structured. **JSONB** lets the backend store a normalized academic payload without constantly changing relational tables when the portal changes.

FOLLOW-UP QUESTION 1:

> **What is the downside of JSONB?**

FOLLOW-UP ANSWER 1:

It is harder to query and **index** compared with normalized tables. For example, finding all students below 75 percent attendance requires parsing **JSONB** in application code right now. That is fine for small data, but not ideal for analytics at scale.

FOLLOW-UP QUESTION 2:

> **When would you normalize it?**

FOLLOW-UP ANSWER 2:

I would normalize fields that become query-heavy: subject code, attendance percentage, marks, SGPA, CGPA, and last sync time. The full **JSONB** blob can still remain as the raw academic snapshot.

FOLLOW-UP QUESTION 3:

> **How would you index JSONB?**

FOLLOW-UP ANSWER 3:

If I had to query inside **JSONB**, I would consider GIN indexes or generated columns for specific JSON paths. But for predictable dashboard and alert queries, extracted relational summary tables are usually cleaner.

Conversation 12: Prisma And Repositories

QUESTION:

> **Why use Prisma in the Express backend?**

ANSWER:

Prisma gives a clean **ORM** layer for **Postgres** and makes relation queries, upserts, and schema visibility easier. The **Prisma** schema clearly documents students, proctors, parents, and mappings. It also keeps most database operations readable for a project with many domain flows.

FOLLOW-UP QUESTION 1:

> **Where do you still use raw SQL?**

FOLLOW-UP ANSWER 1:

FastAPI RAG uses raw **SQL** through psycopg2 to fetch student and proctor mapping data for vector sync. That is acceptable there because **FastAPI** is outside **Prisma**, but I would still wrap it with better connection management and monitoring in production.

FOLLOW-UP QUESTION 2:

> **Are repositories used consistently?**

FOLLOW-UP ANSWER 2:

Not fully. **Auth** and proctor flows use repositories, but the admin controller talks directly to **Prisma** in many places. It works, but I would move that logic into services/repositories to keep controller responsibilities cleaner.

FOLLOW-UP QUESTION 3:

> **How do migrations fit in?**

FOLLOW-UP ANSWER 3:

The schema lives in `prisma/schema.prisma`, and migrations define the actual database changes. In production, migrations should run in **CI/CD** before the app version that depends on them is deployed.

Conversation 13: Query Optimization

QUESTION:

> **How would you optimize the proctor dashboard query?**

ANSWER:

The dashboard fetches mappings by `proctor_id` and `academic_year`, then includes student data. I would add a composite **index** on `(proctor_id, academic_year)` because that matches the filter. I would also avoid recalculating dashboard summaries from **JSONB** on every request once data grows.

FOLLOW-UP QUESTION 1:

> **Why is the existing unique index not enough?**

FOLLOW-UP ANSWER 1:

The existing unique **index** is on `(student_id, academic_year)`, which helps enforce assignment uniqueness. But the dashboard query filters by proctor and year, so the database needs an **index** that matches that access pattern.

FOLLOW-UP QUESTION 2:

> **What would you precompute?**

FOLLOW-UP ANSWER 2:

I would precompute lowest attendance, semester, section, and maybe risk status after each scrape. Then the dashboard can read a small summary instead of parsing every student's **JSONB** blob on every request.

FOLLOW-UP QUESTION 3:

> **How would you prove the index helps?**

FOLLOW-UP ANSWER 3:

I would run `EXPLAIN ANALYZE` before and after adding the **index** on realistic data. Then I would compare query plan, scan type, rows scanned, and response latency under load.

Conversation 14: Scraping Flow

QUESTION:

> **Explain the scraping workflow.**

ANSWER:

The scraper uses **Puppeteer** to log into the college parent portal with USN and DOB. After login, it extracts cookies, closes the browser, and uses **Axios** with those cookies to fetch attendance, CIE, and exam history pages. **Cheerio** parses the HTML, then the normalizer builds a consistent student record and upserts it into **Postgres**.

FOLLOW-UP QUESTION 1:

> **Why not use Puppeteer for everything?**

FOLLOW-UP ANSWER 1:

Puppeteer is heavier because it runs a browser. It is useful for login and dynamic navigation, but once I have cookies, normal HTTP requests are cheaper and faster. That makes scraping more efficient.

FOLLOW-UP QUESTION 2:

> **What if the portal HTML changes?**

FOLLOW-UP ANSWER 2:

The scraper can break because selectors and table structures may change. I tried to make some parsing defensive, but in production I would add HTML fixtures, parser tests, better error reporting, and alerts when scraping success drops.

FOLLOW-UP QUESTION 3:

> **How would you scale scraping?**

FOLLOW-UP ANSWER 3:

I would move scraping into a worker queue with concurrency limits. **API** calls should enqueue a scrape job and return job status instead of blocking while **Puppeteer** runs. That protects the **API** and avoids overwhelming the portal.

Conversation 15: Data Normalization

QUESTION:

> **What does your data normalizer do?**

ANSWER:

It converts scraped portal data into a stable backend format. It standardizes assessment names like T1, T2, AQ1, and AQ2, calculates attendance percentage, derives current year from semester, builds subject records with marks and attendance, and keeps exam history and CGPA in a consistent shape.

FOLLOW-UP QUESTION 1:

> **Why normalize at all?**

FOLLOW-UP ANSWER 1:

Because the frontend, reports, alerts, and AI services need predictable data. If every feature parsed raw portal HTML or inconsistent objects, the system would become fragile very quickly.

FOLLOW-UP QUESTION 2:

> **How do you handle missing data?**

FOLLOW-UP ANSWER 2:

The normalizer uses safe defaults like zero or `N/A` and skips invalid marks where needed. That keeps the dashboard from crashing. But I would still log missing-data patterns so we know when the scraper is failing silently.

FOLLOW-UP QUESTION 3:

> **Where should validation happen?**

FOLLOW-UP ANSWER 3:

Validation should happen at multiple layers: input validation at **API** boundaries, normalization validation after scraping, and schema validation before writing to the database. Right now some of this is manual, and I would strengthen it with formal schemas.

Conversation 16: Report Update And Cooldown

QUESTION:

> **How does manual report update work?**

ANSWER:

`POST /api/report/update` receives a USN, checks the **session**, loads the student, and reads `details.last_updated`. If the last update was within five minutes, it returns 429 with `nextAllowedAt`. Otherwise it triggers a fresh scrape, syncs the data, and returns the updated student dashboard data.

FOLLOW-UP QUESTION 1:

> **Is that full rate limiting?**

FOLLOW-UP ANSWER 1:

No, it is a feature-specific cooldown. It protects the scraping endpoint from rapid repeated updates for the same student, but it does not rate-limit login, admin APIs, **RAG** chat, or general traffic.

FOLLOW-UP QUESTION 2:

> **Why use 429?**

FOLLOW-UP ANSWER 2:

429 means too many requests, and it fits the cooldown behavior. The response also includes `nextAllowedAt`, so the frontend can show a countdown instead of just failing vaguely.

FOLLOW-UP QUESTION 3:

> **What race condition can happen?**

FOLLOW-UP ANSWER 3:

Two update requests for the same student could both pass the cooldown check before either writes `last_updated`. I would fix that with a per-student lock or queue-level de-duplication.

Conversation 17: AI Remark Generation

QUESTION:

> **How does AI remark generation work?**

ANSWER:

Express loads the student's academic data from **Postgres**, extracts the subjects array, and calls **FastAPI** `/generate-remark`. **FastAPI** validates the payload, builds a strict **prompt**, calls **Groq**, and returns a concise academic remark with metadata like model, token usage, and generation time.

FOLLOW-UP QUESTION 1:

> **Why call FastAPI from Express?**

FOLLOW-UP ANSWER 1:

Express already owns the authenticated product flow and knows which student report is being generated. **FastAPI** owns the AI implementation. So **Express** prepares trusted student data and delegates AI generation to the Python service.

FOLLOW-UP QUESTION 2:

> **What if FastAPI is down?**

FOLLOW-UP ANSWER 2:

The **Express** report controller catches connection errors and returns a service-unavailable style response for AI analysis. In production I would add retries for transient failures and a **circuit breaker** so requests fail fast when the AI service is unhealthy.

FOLLOW-UP QUESTION 3:

> **How would you test AI remarks?**

FOLLOW-UP ANSWER 3:

I would mock the **Groq** provider and test **prompt** construction, input validation, and response shape. I would not depend on real **LLM** output in unit tests because it is nondeterministic and slower.

Conversation 18: FastAPI RAG Sync

QUESTION:

> **Explain the RAG sync flow.**

ANSWER:

When a proctor logs in, **Express** calls **FastAPI** `/api/rag/sync`. **FastAPI** starts a background sync task. The **RAG** service reads student records and proctor mappings from **Postgres**, chunks each student into identity, academics, attendance, history, conduct, and misc documents, generates **embeddings** with **Gemini**, and stores them in **PGVector**.

FOLLOW-UP QUESTION 1:

> **Why chunk by topic?**

FOLLOW-UP ANSWER 1:

Topic-based chunks make retrieval more precise. If the question is about attendance, the retriever can focus on attendance chunks instead of pulling a huge mixed student record with unrelated marks and profile data.

FOLLOW-UP QUESTION 2:

> **What is risky about the sync implementation?**

FOLLOW-UP ANSWER 2:

It drops the existing **PGVector** tables before adding new documents. That is simple for a clean rebuild, but risky in production because queries during sync can return no context or fail if the sync crashes halfway.

FOLLOW-UP QUESTION 3:

> **How would you improve sync?**

FOLLOW-UP ANSWER 3:

I would use versioned collections or document-level upserts. Build the new vector set separately, then switch traffic to it after success. I would also store sync state in **Redis** or **Postgres** instead of only in memory.

Conversation 19: RAG Chat Query Flow

QUESTION:

> **How does RAG chat answer a proctor question?**

ANSWER:

FastAPI receives the question and proctor ID. It rewrites the query, detects intent based on keywords, retrieves context using a mix of **BM25** and **PGVector** semantic search, filters documents by proctor ID, formats the retrieved chunks, and asks **Gemini** to answer only from that context.

FOLLOW-UP QUESTION 1:

> **Why combine BM25 and vector retrieval?**

FOLLOW-UP ANSWER 1:

BM25 is good for exact keyword matches like USN, subject codes, or names. Vector retrieval is better for semantic questions. Combining both gives better coverage than relying on only one retrieval style.

FOLLOW-UP QUESTION 2:

> **Is the current RAG endpoint secure?**

FOLLOW-UP ANSWER 2:

Not enough. It filters by `proctor_id`, but that value comes from the request body. If **FastAPI** is exposed, someone could spoof another proctor ID. The safer design is to route **RAG** chat through **Express** after **session** validation.

FOLLOW-UP QUESTION 3:

> **What would you optimize in query time?**

FOLLOW-UP ANSWER 3:

The **BM25** retriever is built from proctor documents during the query flow, which can become expensive. I would prebuild or cache retrievers, or store searchable summaries, especially for proctors with many students.

Conversation 20: Express To FastAPI Communication

QUESTION:

> **How do Express and FastAPI communicate?**

ANSWER:

Express uses an **Axios** client configured with `FASTAPI_URL`, JSON headers, and a 15-second timeout. It calls `/generate-remark` for AI remarks and `/api/rag/sync` for background **RAG** sync notification.

FOLLOW-UP QUESTION 1:

> **Why set a timeout?**

FOLLOW-UP ANSWER 1:

Without a timeout, **Express** requests can hang too long if **FastAPI** or an **LLM** provider is slow. A timeout gives the **API** a bounded failure mode and lets the frontend receive a clear error.

FOLLOW-UP QUESTION 2:

> **Would you make this asynchronous?**

FOLLOW-UP ANSWER 2:

For AI remarks, synchronous can be acceptable if the response is quick. For **RAG** sync, scraping, PDF generation, and email delivery, I would prefer background jobs because they are heavier and less predictable.

FOLLOW-UP QUESTION 3:

> **How would you protect service-to-service calls?**

FOLLOW-UP ANSWER 3:

I would keep **FastAPI** private on the internal network and require a service token or signed request from **Express**. The frontend should not need to call **FastAPI** directly for sensitive data.

Conversation 21: Notifications

QUESTION:

> **How are proctor notifications generated?**

ANSWER:

The backend fetches all students assigned to a proctor for the academic year, parses each student's **JSONB** details, extracts subjects, finds subjects below 75 percent attendance, sorts them by lowest attendance, and returns alert data to the UI.

FOLLOW-UP QUESTION 1:

> **Why compute notifications dynamically?**

FOLLOW-UP ANSWER 1:

For a small dataset it is simple and always reflects the latest student details. It avoids managing another alerts table early in the project.

FOLLOW-UP QUESTION 2:

> **Why is this a scalability issue?**

FOLLOW-UP ANSWER 2:

Because every notification request may scan multiple students and parse **JSONB** in Node. With many proctors and students, that becomes CPU-heavy and slower than serving precomputed alert rows.

FOLLOW-UP QUESTION 3:

> **How would you improve it?**

FOLLOW-UP ANSWER 3:

I would compute alerts after each scrape or sync and store them in an indexed table. Then the notifications endpoint becomes a simple query by proctor ID and academic year.

Conversation 22: PDF And Email Reports

QUESTION:

> **How does report email delivery work?**

ANSWER:

The frontend sends the report HTML and USN to **Express**. The backend fetches student and parent records, uses **Puppeteer** to render the HTML into an A4 PDF, uploads the PDF to **Cloudinary**, then sends it to each parent through **Resend** while tracking success or failure per parent.

FOLLOW-UP QUESTION 1:

> **Why generate PDF server-side?**

FOLLOW-UP ANSWER 1:

Server-side PDF generation gives more control and consistency. It also lets the backend attach the same official report to emails without depending on the client's browser environment.

FOLLOW-UP QUESTION 2:

> **What is risky about accepting HTML from the frontend?**

FOLLOW-UP ANSWER 2:

Rendering client-provided HTML in **Puppeteer** can be risky if it includes unwanted scripts or external resources. For production, I would sanitize it or generate reports server-side from trusted templates and data.

FOLLOW-UP QUESTION 3:

> **Should email sending be synchronous?**

FOLLOW-UP ANSWER 3:

For a small number of parents it works, but production should use a queue. PDF generation, upload, and email delivery are slow external operations, and the **API** should return job status instead of making the user wait.

Conversation 23: WhatsApp Delivery

QUESTION:

> **How does WhatsApp report delivery work?**

ANSWER:

The backend generates the report PDF, uploads it to **Cloudinary**, builds a message for each parent, and sends it using **Twilio WhatsApp** if **Twilio** credentials are configured. If **Twilio** is not configured, it still returns prepared message data and status.

FOLLOW-UP QUESTION 1:

> **How do you handle phone numbers?**

FOLLOW-UP ANSWER 1:

The code cleans spaces and symbols and defaults to the Indian country code if no plus prefix is present. That is okay locally, but production should use a proper phone validation library and store normalized E.164 numbers.

FOLLOW-UP QUESTION 2:

> **What if one parent's message fails?**

FOLLOW-UP ANSWER 2:

The result is captured per parent. One failure should not block every other parent. I would also store delivery attempts and retry failed sends through a background worker.

FOLLOW-UP QUESTION 3:

> **What production concern exists with Cloudinary links?**

FOLLOW-UP ANSWER 3:

Reports contain sensitive academic data, so public or long-lived links need care. I would use access-controlled storage, expiring signed URLs, or stricter permissions depending on the compliance requirements.

Conversation 24: Admin Backend

QUESTION:

> **Explain the admin backend.**

ANSWER:

The admin routes manage proctors, student assignments, parent details, unassigned students, and basic stats. They use **Prisma** operations like `findMany`, `upsert`, `delete`, and `count`. The assignment design uses `proctor_student_map` with academic year to keep proctor-student assignment history clean.

FOLLOW-UP QUESTION 1:

> **What is the biggest admin issue?**

FOLLOW-UP ANSWER 1:

The backend admin routes are not protected. The frontend has a client-side admin password, but that does not secure the **API**. Production needs real backend admin **authentication** and **authorization**.

FOLLOW-UP QUESTION 2:

> **What transaction issue exists?**

FOLLOW-UP ANSWER 2:

Bulk assignment loops through students one by one, so partial updates can happen if one write fails. Proctor deletion also deletes mappings and then the proctor. I would wrap these multi-step operations in **Prisma transactions**.

FOLLOW-UP QUESTION 3:

> **What bug risk exists in removing students?**

FOLLOW-UP ANSWER 3:

The remove-student flow calculates the proctor ID but deletes by student and academic year. It should also verify the mapping belongs to that proctor, otherwise the URL proctor ID is not really being enforced.

Conversation 25: Validation And Error Handling

QUESTION:

> **How do you handle validation and errors?**

ANSWER:

Validation is currently mostly manual in controllers: required fields are checked and errors are returned with status codes. **FastAPI** has Pydantic models available, but some routes accept raw dictionaries. Error handling uses try/catch in controllers and a global **Express** error handler, but it is not fully standardized yet.

FOLLOW-UP QUESTION 1:

> **What would you improve in validation?**

FOLLOW-UP ANSWER 1:

I would use Zod or **express**-validator in **Express** and Pydantic models consistently in **FastAPI**. That would make request bodies, params, and query validation explicit and easier to test.

FOLLOW-UP QUESTION 2:

> **What is wrong with inconsistent error shapes?**

FOLLOW-UP ANSWER 2:

The frontend has to handle too many response formats, and debugging becomes harder. A consistent error shape with code, message, request ID, and details is cleaner for both developers and clients.

FOLLOW-UP QUESTION 3:

> **Should all errors go through the global handler?**

FOLLOW-UP ANSWER 3:

Most unexpected errors should. Expected domain errors can still return specific status codes, but I would standardize them through custom error classes so controllers do not repeat response logic everywhere.

Conversation 26: Security Hardening

QUESTION:

> **What security improvements would you make first?**

ANSWER:

First I would fix **authorization**: backend admin **auth**, role checks, **ownership checks**, and **FastAPI** protection. Then I would restrict **CORS**, move **session** storage to secure **httpOnly** cookies or harden token storage, add login rate limiting, sanitize report HTML, and audit sensitive admin actions.

FOLLOW-UP QUESTION 1:

> **Why are session IDs in localStorage risky?**

FOLLOW-UP ANSWER 1:

If the app has an **XSS** bug, JavaScript can read localStorage and steal the **session** ID. **HttpOnly** cookies reduce that risk because scripts cannot read them directly.

FOLLOW-UP QUESTION 2:

> **Is CORS a security boundary?**

FOLLOW-UP ANSWER 2:

Not by itself. **CORS** mainly controls browser-based cross-origin access. It does not stop server-to-server requests. Real security still needs **authentication** and **authorization** on the backend.

FOLLOW-UP QUESTION 3:

> **How would you protect login?**

FOLLOW-UP ANSWER 3:

I would rate-limit by IP and identity, avoid user enumeration in error messages, monitor failed attempts, enforce stronger proctor/admin passwords, and consider OTP or institutional SSO for stronger identity assurance.

Conversation 27: Scalability

QUESTION:

> **What would fail first under heavy traffic?**

ANSWER:

The scraper would fail first because **Puppeteer** is CPU and memory heavy and depends on an external college portal. After that, **JSONB** parsing in dashboard and notification APIs could become slow, and **RAG** sync could become expensive because it rebuilds vector data.

FOLLOW-UP QUESTION 1:

> **How would you scale Express?**

FOLLOW-UP ANSWER 1:

I would run multiple **Express** instances behind a load balancer. Since **sessions** are in **Redis** and data is in **Postgres**, requests do not need sticky **sessions**. I would also move CPU-heavy work to workers.

FOLLOW-UP QUESTION 2:

> **How would you handle one million users?**

FOLLOW-UP ANSWER 2:

I would separate request/response APIs from background jobs, add caching, add DB indexes, use read replicas for read-heavy endpoints, scale **Express** and **FastAPI** independently, and make scraping queue-based with strict concurrency limits.

FOLLOW-UP QUESTION 3:

> **What should be cached first?**

FOLLOW-UP ANSWER 3:

Proctor dashboard summaries, notification alerts, and AI remarks keyed by student `last_updated`. Those are read-heavy and do not need to be recomputed on every request if the underlying student data has not changed.

Conversation 28: Caching Strategy

QUESTION:

> **What caching strategy would you use?**

ANSWER:

Redis is already in the stack, so I would use it for short-lived caches. Dashboard summaries can be cached by proctor ID and academic year. AI remarks can be cached by student USN plus `last_updated`. Notifications can be cached until a scrape updates student details or assignments change.

FOLLOW-UP QUESTION 1:

> **How do you avoid cache leaks?**

FOLLOW-UP ANSWER 1:

Cache keys must include ownership context, like proctor ID and academic year. Also, **authorization** should happen before returning cached data. I would never use a generic key like just `dashboard` for user-specific data.

FOLLOW-UP QUESTION 2:

> **How do you invalidate caches?**

FOLLOW-UP ANSWER 2:

After a successful scrape, invalidate that student's report and any proctor dashboards or notifications affected by that student. After assignment changes, invalidate proctor dashboard and alert caches for the affected academic year.

FOLLOW-UP QUESTION 3:

> **What should not be cached?**

FOLLOW-UP ANSWER 3:

Sensitive responses should not be cached in shared layers unless the key and access control are very strict. I also would not cache failed **auth** responses or anything that depends on rapidly changing **session** state.

Conversation 29: Concurrency And Race Conditions

QUESTION:

> **What race conditions can happen in this backend?**

ANSWER:

Two update requests for the same student can both pass the cooldown and trigger duplicate scrapes. **RAG** sync uses an in-memory lock, so multiple **FastAPI** workers could sync at the same time. Bulk assignment can partially succeed if one write fails mid-loop.

FOLLOW-UP QUESTION 1:

> **How would you prevent duplicate scrapes?**

FOLLOW-UP ANSWER 1:

I would use a per-student job key in a queue. If a job for that USN is already pending or running, return the existing job status instead of starting another scrape.

FOLLOW-UP QUESTION 2:

> **How would you make RAG sync safe?**

FOLLOW-UP ANSWER 2:

Use a distributed lock in **Redis** or **Postgres** and make sync idempotent. I would also avoid dropping live vector tables and instead sync into a new version before switching.

FOLLOW-UP QUESTION 3:

> **How would you handle partial bulk assignment?**

FOLLOW-UP ANSWER 3:

I would decide the business behavior first. If it should be all-or-nothing, wrap it in a transaction. If partial success is acceptable, return a multi-status response with success and failure arrays.

Conversation 30: Deployment And Docker

QUESTION:

> **How would you deploy this backend?**

ANSWER:

Right now the project uses local batch scripts, not **Docker**. For production I would containerize **Express** and **FastAPI** separately, run them behind a **reverse proxy** or load balancer, keep **FastAPI** private, use **managed Postgres** and **Redis**, run **Prisma** migrations in **CI/CD**, and configure **health checks**.

FOLLOW-UP QUESTION 1:

> **Why containerize the services separately?**

FOLLOW-UP ANSWER 1:

Express and **FastAPI** have different runtimes, dependencies, and scaling needs. Separate containers let me scale the product **API** and AI service independently and keep deployments cleaner.

FOLLOW-UP QUESTION 2:

> **What special issue does Puppeteer create in Docker?**

FOLLOW-UP ANSWER 2:

Puppeteer needs Chromium and system dependencies. The container image must include those dependencies, and memory/concurrency limits need to be tuned because browser automation is heavier than normal **API** code.

FOLLOW-UP QUESTION 3:

> **What health checks would you add?**

FOLLOW-UP ANSWER 3:

I would keep simple liveness checks, but add readiness checks for **Postgres** and **Redis** on **Express**, and model/config/database readiness on **FastAPI**. That prevents routing traffic to a service that is running but not actually ready.

Conversation 31: Logging And Monitoring

QUESTION:

> **How would you monitor this in production?**

ANSWER:

I would track **API** latency, error rates, **Redis** availability, **Postgres** query latency, scrape duration and failure rate, **FastAPI** latency, **LLM** errors, token usage, **RAG** sync status, email/**WhatsApp** delivery failures, and queue depth once background jobs are introduced.

FOLLOW-UP QUESTION 1:

> **What logging change would you make first?**

FOLLOW-UP ANSWER 1:

I would replace scattered `console.log` and `print` statements with structured logs. Each request should have a request ID so I can trace a report request from **Express** to **FastAPI** and external providers.

FOLLOW-UP QUESTION 2:

> **What alert would matter most?**

FOLLOW-UP ANSWER 2:

Scrape failure rate is critical because fresh student data depends on it. I would also alert on **Redis** failures because **auth** depends on **Redis**, and **FastAPI** failures because AI features depend on it.

FOLLOW-UP QUESTION 3:

> **How would you debug a slow report request?**

FOLLOW-UP ANSWER 3:

I would trace each stage: **auth**/**session** check, database read, scraper if triggered, **FastAPI** call if AI is requested, PDF generation if email is involved, and external provider latency. **Request IDs** and timing logs make that much easier.

Conversation 32: Testing Strategy

QUESTION:

> **How would you test this backend?**

ANSWER:

I would start with unit tests for pure logic: DOB formatting, scraper normalization, **prompt** building, and **session** helpers. Then integration tests for **Express** routes using a test **Postgres** and **Redis**. For **FastAPI**, I would test validation and mock **Groq**/**Gemini** providers. For scraping, I would use saved HTML fixtures instead of hitting the real portal in tests.

FOLLOW-UP QUESTION 1:

> **What would be your first test?**

FOLLOW-UP ANSWER 1:

I would test DOB normalization and student login behavior because **auth** is central and date formats are easy to break. Then I would add tests around **session** middleware and protected route access.

FOLLOW-UP QUESTION 2:

> **How do you test Puppeteer scraping?**

FOLLOW-UP ANSWER 2:

I would separate parsing from browser automation. Parser logic can be tested with saved HTML fixtures. Browser login can have a smaller smoke/integration test, but I would not make every CI run depend on the live college portal.

FOLLOW-UP QUESTION 3:

> **How do you test AI code?**

FOLLOW-UP ANSWER 3:

I would mock the **LLM** providers. The tests should verify **prompt** construction, input validation, error handling, and response shape. Real model quality is better evaluated separately with sample datasets, not normal unit tests.

Conversation 33: Debugging Student Login

QUESTION:

> **A student says login fails even with correct credentials. How would you debug?**

ANSWER:

I would first inspect the request payload and DOB format. Then I would check whether the student exists in **Postgres** with that USN and normalized DOB. If not, I would check whether the fallback scraper ran and whether it failed due to portal login, network, or parsing issues.

FOLLOW-UP QUESTION 1:

> **What is the most likely simple bug?**

FOLLOW-UP ANSWER 1:

DOB format mismatch. The frontend might send one format while the database has another. That is why `formatDOB` is important and why I would log normalized DOB during debugging without exposing sensitive data publicly.

FOLLOW-UP QUESTION 2:

> **What if scraping fails?**

FOLLOW-UP ANSWER 2:

Then login should return invalid credentials, but internally I would inspect scraper logs, portal response, timeout, and whether the page structure changed. I would also check if **Puppeteer** can launch in the current environment.

FOLLOW-UP QUESTION 3:

> **How would you make this easier to debug later?**

FOLLOW-UP ANSWER 3:

I would add structured **auth** logs with **request IDs** and failure reasons classified internally, like DOB mismatch, user missing, scraper timeout, or portal login failed. The client can still receive a generic safe message.

Conversation 34: Debugging RAG Chat

QUESTION:

> **The RAG chatbot gives empty or wrong answers. How would you debug?**

ANSWER:

I would check sync status first. Then I would confirm the proctor has mapped students, inspect whether chunks were generated, verify **PGVector** documents have the correct metadata, and look at the retrieved context before it goes to **Gemini**.

FOLLOW-UP QUESTION 1:

> **What if sync status says never synced?**

FOLLOW-UP ANSWER 1:

Then the chatbot may have no vector context. I would trigger `/api/rag/sync`, check **FastAPI** logs, database connectivity, embedding **API** errors, and whether **PGVector** insertion succeeded.

FOLLOW-UP QUESTION 2:

> **What if the wrong student's data appears?**

FOLLOW-UP ANSWER 2:

That points to **metadata filtering** or **authorization**. I would verify that every chunk has the correct `proctor_id`, and I would also fix the design so the proctor ID comes from a validated **session**, not from the request body.

FOLLOW-UP QUESTION 3:

> **How do you evaluate RAG quality?**

FOLLOW-UP ANSWER 3:

I would create a small set of known proctor questions with expected answers, then measure whether retrieval returns the right chunks and whether the final answer stays grounded. **RAG** debugging starts with retrieval, not just the final **LLM** text.

Conversation 35: Final Production Readiness

QUESTION:

> **Is this backend production-ready?**

ANSWER:

Functionally, it has strong backend pieces, but I would not call it fully production-ready yet. The main missing parts are **authorization** hardening, protected admin routes, **FastAPI** isolation, background queues, **Docker**, **structured logging**, tests, stricter validation, and operational monitoring.

FOLLOW-UP QUESTION 1:

> **What would you fix first?**

FOLLOW-UP ANSWER 1:

Security first: admin backend **auth**, role checks, **ownership checks**, and removing direct **FastAPI** access from the browser. Scaling does not matter if the system can leak or modify the wrong data.

FOLLOW-UP QUESTION 2:

> **What would you tell an interviewer honestly?**

FOLLOW-UP ANSWER 2:

I would say the project is a strong backend prototype with real workflows, but I know exactly what needs hardening before production. That is a better answer than pretending every part is already enterprise-grade.

FOLLOW-UP QUESTION 3:

> **What is your strongest defense of the project?**

FOLLOW-UP ANSWER 3:

It solves a real backend problem end to end: **auth**, **sessions**, database modeling, scraping, normalization, reports, parent communication, AI remarks, and **RAG** chat. The architecture is explainable, and the tradeoffs are clear.

3. Things To Revise Before Interview

Most Important APIs

- `POST /api/auth/login`
- `POST /api/auth/proctor-login`
- `GET /api/auth/profile`
- `GET /api/report/student/:usn`
- `GET /api/report/:usn`
- `POST /api/report/update`
- `POST /api/report/send-email`
- `POST /api/report/send-whatsapp`
- `GET /api/proctor/:proctorId/dashboard`
- `GET /api/proctor/:proctorId/scrape-list`
- `GET /api/proctor/:proctorId/student/:studentUsn`
- `GET /api/notifications/:proctorId`
- `POST /api/students/sync`
- `/api/admin/*`
- `POST /generate-remark`
- `POST /api/rag/sync`
- `GET /api/rag/sync/status`
- `POST /api/rag/chat`

Most Important Backend Flows

- Student login -> **Redis session** -> dashboard.
- Missing student data -> scraper -> normalization -> **Postgres JSONB**.

- Manual report update -> cooldown -> scraper.
- AI report -> **Express** -> **FastAPI** -> **Groq**.
- Proctor login -> **Redis session** -> **RAG** sync notification.
- Proctor dashboard -> assignments -> **JSONB** parsing -> lowest attendance.
- Notifications -> assigned students -> low attendance detection.
- Email report -> **Puppeteer** PDF -> **Cloudinary** -> **Resend**.
- **WhatsApp** report -> PDF -> **Cloudinary** -> **Twilio**/prepared messages.
- **RAG** chat -> query rewrite -> intent detection -> **BM25** plus **PGVector** -> **Gemini**.

Authentication Flow To Remember

- This is not **JWT**.
- **Session** ID is a random **UUID**.
- **Session** ID is passed as `x-session-id`.
- **Redis** stores `session:<id>` to `student:<USN>` or `proctor:<ID>`.
- **Session TTL** is 30 days and refreshed in middleware.
- Logout deletes **session** and reverse mapping.
- Proctor passwords are **bcrypt** hashed.
- Student login uses USN plus DOB.

Middleware Chain To Revise

- `cors()`
- `express.json()`
- route-specific `requireSession`
- router-level `verifySession` for proctor and notifications
- global `errorHandler`

Important DB Relationships

- `Student.usn` is **primary key**.
- `Proctor.proctor_id` is **primary key**.
- `Parent` uses composite key `(usn, relation)`.
- `ProctorStudentMap` joins proctors and students.
- Unique assignment: `(student_id, academic_year)`.
- `Student.details` is **JSONB** academic payload.

Important Service Interactions

- **Express** -> **Redis** for **sessions**.
- **Express** -> **Prisma**/**Postgres** for app data.
- **Express** -> **FastAPI** for AI remark and **RAG** sync.
- **FastAPI** -> **Postgres** for **RAG** data and **PGVector**.

- **Express** -> college portal through **Puppeteer**/**Axios**.
- **Express** -> **Cloudinary** and **Resend** for report delivery.
- **Express** -> **Twilio** for **WhatsApp** if configured.
- Frontend -> **FastAPI** directly for **RAG** chat currently.

Important Async Flows

- **Puppeteer** login and navigation.
- **Axios** parallel fetches for unique portal URLs.
- Student sync loop with **Prisma** upserts.
- Fire-and-forget **RAG** sync notification.
- **FastAPI** background task for **RAG** sync.
- Sequential extension-driven batch scraping.
- Sequential parent email sending with per-parent result capture.

Docker And Deployment Points

- No **Dockerfile** or **docker-compose** currently exists.
- Local startup uses `start-all.bat` / `run.bat`.
- **Express** default port is `5000`, but frontend defaults to `5001`; **Express** `.env` must set `PORT=5001` for local dev to work.
- **FastAPI** runs on `127.0.0.1` in `main.py`.
- Production should containerize **Express** and **FastAPI** separately.
- **FastAPI** should be private behind **Express** or internal networking.
- Run **Prisma** migrations in **CI/CD**.
- Add **health checks** for **Express**, **FastAPI**, **Redis**, **Postgres**.

Common Interviewer Traps

- Do not say **JWT**. Say **Redis**-backed opaque **sessions**.
- Do not call it pure microservices. Say two-service architecture or distributed monolith.
- Do not claim admin **auth** is production-secure.
- Do not claim **FastAPI RAG** is protected.
- Do not claim **Docker** is implemented.
- Do not claim full rate limiting exists.
- Do not claim there is strong automated testing.
- Do not hide the **authorization** gap.
- Do not say **JSONB** is always better than relational tables.
- Do not say scraping is scalable as-is.

Weak Areas Interviewers May Attack

- Public admin backend routes.
- Client-side admin password.

- Direct frontend calls to **FastAPI RAG**.
- Missing role and ownership **authorization**.
- **Session** IDs in localStorage.
- No **CSRF**/**XSS** hardening story.
- No queue for scraper/PDF/email.
- No full **rate limiter**.
- No **Docker** production setup.
- No automated tests.
- Legacy schema files: `init_postgres.js`, duplicate student routes. Note: `seed.js` is still referenced in `prisma.config.ts` and `package.json`.
- **RAG** sync drops vector tables.

Production Improvements To Mention Confidently

- Add `requireRole` and ownership middleware.
- Protect admin APIs with backend **auth**.
- Proxy all **FastAPI** calls through **Express** or **service-to-service auth**.
- Move scraping to **Redis** queue workers.
- Move PDF/email/**WhatsApp** to background jobs.
- Add DB **index** on `(proctor_id, academic_year)`.
- Add **transactions** for bulk assignment and proctor deletion.
- Add **structured logging** and **request IDs**.
- Add centralized validation.
- Add Dockerfiles and **CI/CD**.
- Add test suite with mocked external services.
- Restrict **CORS**.
- Use **httpOnly secure cookies** or stronger token storage.

Backend Terms To Use

- **opaque session ID**
- **sliding TTL**
- **RBAC**
- ownership **authorization**
- **JSONB**
- **composite unique constraint**
- transaction
- idempotency
- **connection pooling**
- background worker
- queue
- **retry with backoff**

- **circuit breaker**
- **structured logging**
- request correlation ID
- metadata-filtered retrieval
- **vector store**
- **BM25**
- **semantic retrieval**
- materialized summary

4. Ready-Made Interview Explanations

2-Minute Project Explanation

MSR Insight is an academic reporting and proctoring system. My backend has two services. Express is the main API gateway: it handles student and proctor login, Redis-backed sessions, Prisma/Postgres data access, scraping academic data from the college portal, report APIs, parent email, and WhatsApp workflows. FastAPI is the intelligence service: it generates AI academic remarks using Groq and powers a RAG chatbot using Gemini, LangChain, and PGVector.

The core data model has students, proctors, parents, and a proctor-student mapping per academic year. Student academic data is stored as JSONB because scraped data is semi-structured and changes with the portal. For auth, I used Redis-backed session IDs passed through `x-session-id`, not JWT. For scraping, Puppeteer logs into the portal, then Axios and Cheerio fetch and parse the actual pages more efficiently.

If I were taking this to production, my first improvements would be backend admin auth, role and ownership checks, moving scraper/PDF work into queues, adding DB indexes, structured logging, tests, Docker, and protecting FastAPI behind Express.

5-Minute Backend-Focused Explanation

The backend is split into Express and FastAPI. Express owns the product workflow. It starts in `server.js`, wires middleware and routes in `app.js`, and exposes route groups for auth, reports, proctors, notifications, student sync, and admin operations. The auth flow uses Redis-backed opaque sessions. On login, the backend creates a random UUID, stores `session:<id>` in Redis with the user role and identity, and the frontend sends that ID in `x-session-id` for protected requests.

The database is PostgreSQL with Prisma. The main schema has `Student`, `Proctor`, `Parent`, and `ProctorStudentMap`. `ProctorStudentMap` enforces one proctor assignment per student per academic year. The student's academic data is in a JSONB `details` field. I chose that because the source data comes from scraping and includes variable subject, attendance, CIE, and exam-history structures. The tradeoff is that JSONB is less ideal for analytics and filtering, so production would add extracted summary fields and indexes.

The scraping workflow is one of the more backend-heavy parts. For student login or manual update, the backend can trigger Puppeteer to authenticate against the college parent portal. After login it extracts cookies, closes the browser, and switches to Axios plus Cheerio to fetch attendance, CIE, and exam-history

pages. Then a normalizer standardizes assessment labels, attendance percentages, marks, current year, CGPA, and exam history before upserting into Postgres.

FastAPI handles AI workloads. Express calls `/generate-remark` for AI remarks. FastAPI validates the subject list, builds a strict two-line prompt, and calls Groq. For proctor chat, FastAPI has a RAG service. It syncs student records from Postgres, chunks each student into identity, academic, attendance, history, conduct, and misc documents, embeds them using Gemini embeddings, and stores them in PGVector. At query time it rewrites the query, detects intent, combines BM25 and vector retrieval, filters by proctor ID, and sends retrieved context to Gemini.

The honest production answer is that the backend has strong architecture pieces, but the main gaps are authorization and operational hardening. Admin routes are currently public, FastAPI is directly callable, and several routes only validate that a session exists rather than checking role and ownership. Before production I would fix RBAC, protect FastAPI, move scraping and report delivery to background queues, add transactions and indexes, add observability, and containerize the services.

Short Recruiter-Friendly Explanation

I built the backend for an AI-powered academic reporting platform. It has an Express API gateway with Redis sessions, Prisma/Postgres data models, scraping automation, report generation, email/WhatsApp delivery, and a FastAPI AI service for Groq-based remarks and Gemini/LangChain RAG chatbot features.

Strong Backend Contribution Explanation

My strongest contribution was designing the backend flow end to end: session-based auth, student/proctor APIs, the Prisma/Postgres schema, scraping and normalization, report generation, and the Express-to-FastAPI AI integration. I can also explain the production tradeoffs clearly, especially around authorization, scaling Puppeteer, JSONB design, Redis sessions, and RAG data isolation.

5. Live Mock Interview Mode

Use this section when practicing. Ask one question at a time and answer out loud.

Mock Round 1: Warm-Up

Question 1: Explain MSR Insight from a backend perspective in two minutes.

What a strong answer must include:

- **Express** as main **API gateway**.
- **FastAPI** as intelligence service.
- **Redis sessions**, not **JWT**.
- **Prisma/Postgres** with **JSONB**.
- Scraper and report workflows.
- Honest production gaps.

Interviewer push:

"You said two services. Is this actually microservices or just a split backend?"

Mock Round 2: Auth Pressure

Question 2: Walk me through proctor login from HTTP request to Redis session.

Interviewer push:

"After session validation, how do you know that this proctor is allowed to access `:proctorId` in the URL?"

Expected honest answer:

"Currently the middleware validates session existence, but the route needs an additional ownership check. I would add middleware that requires role `proctor` and compares `req.user.id` with `req.params.proctorId`, with admin override where appropriate."

Mock Round 3: Scaling Pressure

Question 3: What fails first if 500 proctors trigger batch scraping together?

Expected answer:

The scraper and external portal fail first. Puppeteer is heavy, requests are synchronous, and the portal may throttle. Move scraping into a queue with concurrency limits, backoff, retries, job status, and duplicate job prevention.

Mock Round 4: Database Pressure

Question 4: Why did you store student academic data in JSONB instead of fully normalized tables?

Expected answer:

Because scraped data is semi-structured and changes. JSONB is flexible for subjects, attendance, CIE, and exam history. But for analytics and scale, extract frequently queried fields and add indexes/materialized summaries.

Mock Round 5: RAG Pressure

Question 5: How do you prevent a proctor from querying another proctor's students in RAG?

Expected answer:

Currently retrieval filters by `proctor_id`, but FastAPI trusts the request body. Production should not expose FastAPI directly. Express should validate the session and ownership, then call FastAPI with a trusted proctor ID or service token.

6. Final Backend Evaluation

Scores

Area	Score	Reason
Backend engineering	7.2/10	Good service layering, **Prisma**, **Redis**, scraping, AI integration
API design	6.8/10	Clear route groups, but inconsistent **auth**/validation/error patterns
Scalability	5.8/10	Good separation, but scraper, **RAG** sync, and **JSONB** parsing need work
Security	4.2/10	Main weakness: admin/**FastAPI** exposure and missing **authorization** checks
Production readiness	4.8/10	No **Docker**, no tests, limited logging, no queues, broad **CORS**
Interview readiness	7.5/10	Strong project if you explain tradeoffs honestly
Resume impact	8.0/10	AI/**RAG** + **Redis sessions** + **Prisma**/**Postgres** + scraper + reports is impressive

Strongest Backend Discussion Areas

- **Express** plus **FastAPI** architecture.
- **Redis**-backed **sessions**.
- **PostgreSQL**/**Prisma** schema.
- **JSONB** tradeoff.
- **Puppeteer** scraping pipeline.
- AI remark generation.
- **RAG** chatbot design.
- Report delivery with PDF, **Cloudinary**, **Resend**, **Twilio**.

Weakest Backend Discussion Areas

- **Authorization**.
- Production security.
- **Docker**/deployment.
- Automated testing.
- Queue-based background processing.
- Transaction handling.
- Observability.
- **FastAPI** endpoint protection.

Most Likely Interviewer Concerns

- "Why are admin APIs public?"
- "How do you stop one proctor from reading another proctor's data?"
- "Why is **FastAPI** directly called from the frontend?"

- "What happens when **Puppeteer** becomes slow?"
- "Why no **Docker**?"
- "Why no tests?"
- "Can this scale beyond a demo?"

Most Impressive Implementation Areas

- The scraper optimization: **Puppeteer** for login, **Axios**/**Cheerio** for page fetches.
- **Redis**-backed **session** lifecycle.
- **RAG** chunking by semantic category.
- Combining **BM25** and **semantic retrieval**.
- **JSONB**-based academic payload storage.
- Parent report pipeline with PDF generation and delivery.

What To Improve Before Interviews

If you can make small code changes before interviews, prioritize:

1. Add backend **auth** to admin routes.
2. Add role and ownership middleware.
3. Stop frontend from calling **FastAPI** directly.
4. Add indexes in **Prisma** schema.
5. Add a few tests for **auth**, DOB formatting, and scraper normalization.
6. Add Dockerfiles for **Express** and **FastAPI**.
7. Clean legacy files or explain them as migration leftovers.

If you cannot change code before interviews, be ready to say:

"The system is functionally strong, but the next production milestone is security and operations: RBAC, ownership checks, FastAPI isolation, queues for heavy work, tests, Docker, and observability."